Kova: arquitectura objetivo y plan de migración

Prueba técnica · Platform / Cloud Engineer

Autor: **Camilo Rozo**

Repositorio (documentos, libros de costos y esqueleto de Terraform):
https://github.com/Cerozob/puruebatecnica-cloudengineer/tree/claude/optimistic-feynman-0baipi

Diagramas editables (Eraser):
https://app.eraser.io/workspace/lj84o5Dz4IH3ScJQkZkG

Contenido

1. Uso de IA generativa 3

2. Arquitectura actual y suposiciones 5

3. Arquitectura objetivo 8

4. Plan de migración y modernización 12

5. Decisiones técnicas y suposiciones 17

6. Estimado de costos y recortes 20

7. Controles de seguridad de los primeros 90 días 22

8. Infraestructura como código 23

Anexo A. Decisiones en detalle 25

1. Uso de IA generativa

1. Herramientas de IA Generativa Utilizadas

- Claude Code (sesión en la nube, con modo de planeación)
- Servidores MCP conectados a Claude Code:
- AWS Knowledge / AWS MCP, para consultar documentación oficial de AWS
- AWS Pricing, para precios unitarios
- Eraser, para escribir los diagramas como código

2. Qué tareas apoyo con IA

- Investigación: verificar contra la documentación de AWS cada servicio y cada afirmación (curvas soportadas por KMS y CloudHSM, alcance de DRS, Nitro Enclaves, VPC Lattice con ECS, novedades como OpenSearch Serverless NextGen y CloudWatch Omni).
- Redacción de borradores: decisiones (ADRs), plan de migración y modernización, controles de seguridad, narrativas de los diagramas y este documento.
- Código de los diagramas en Eraser (solo diagramas como código, sin los generadores de IA de Eraser) y el esqueleto de Terraform.

La numeración de los diagramas, su revisión visual y la exportación las hice yo.

3. Qué validaciones realizo sobre lo generado

- Cada decisión la tomé yo; el agente solo la cuestionaba contra las buenas prácticas de AWS y yo decidía si cambiarla.
- Las afirmaciones técnicas se contrastaron con la documentación oficial de AWS. Cuando la fuente no era accesible desde el agente (por ejemplo, un hilo de re:Post), revisé la fuente yo mismo.
- Los precios clave los revisé en la calculadora de AWS (por ejemplo, el costo de CloudHSM).
- El esqueleto de Terraform se validó con `terraform fmt` y `terraform validate`.

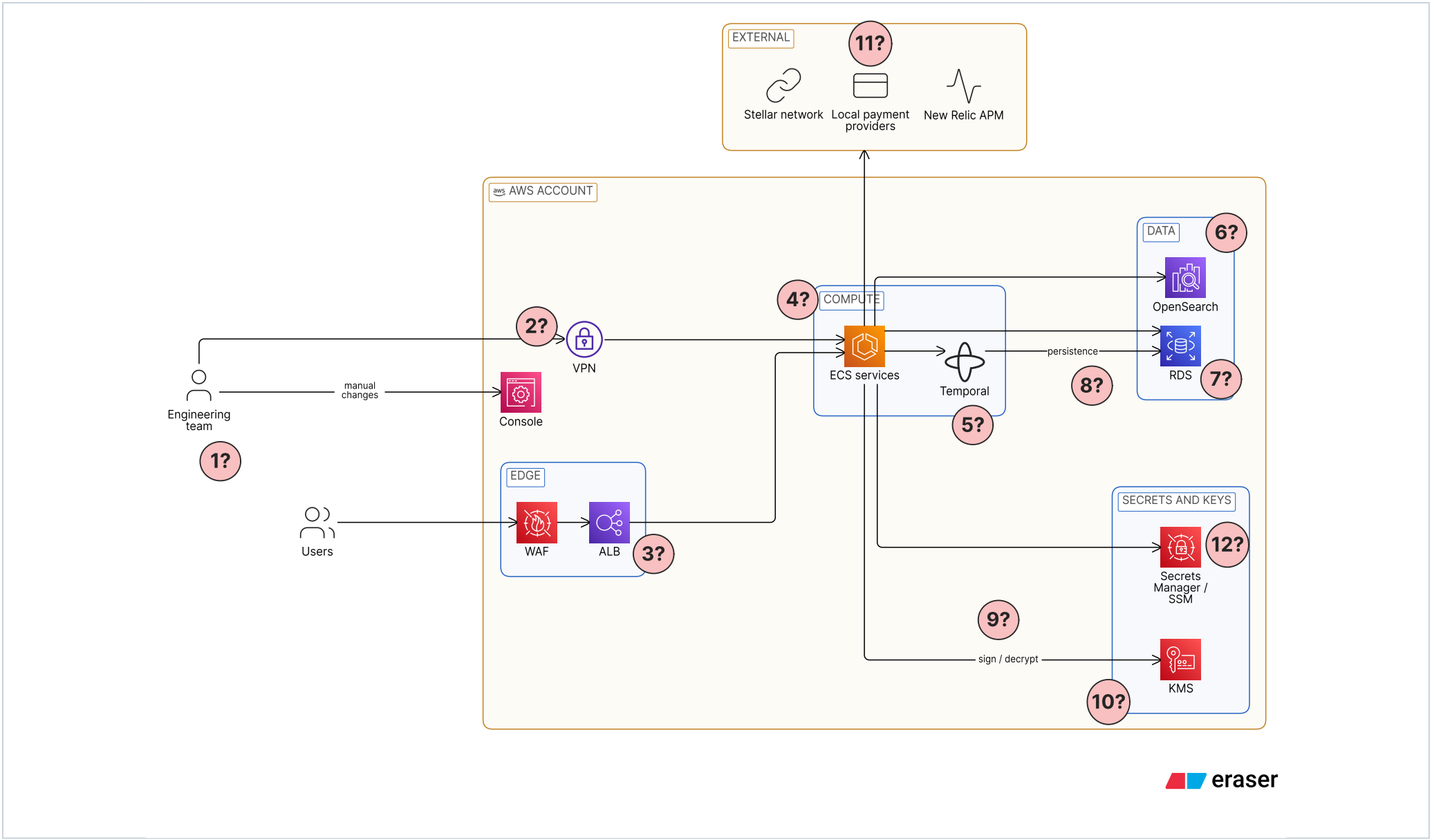
4. Qué riesgos identifiqué

- **Afirmaciones plausibles pero falsas:** por ejemplo, proponer AWS Elastic Disaster Recovery para cargas que no cubre (ECS, RDS). Se detectó al verificar en la documentación.
- **Fuentes mal usadas:** citar un artículo que habla de otra cosa (un blog de análisis de wallets en DynamoDB no trata de custodia de llaves).
- **Sobreingeniería y sobredocumentación:** tendencia a agregar más servicios, detalle y texto del necesario para una prueba de diseño.
- **Desviación:** ante la ambigüedad, el agente asume; por eso las suposiciones quedaron en una lista explícita.

5. Cómo evité depender ciegamente de la herramienta

- Diseñé y decidí yo; el agente actuó como contraparte que cuestiona, con la regla "yo decido, tú me retas".
- Respondí siempre con mis palabras en lugar de aceptar opciones prefabricadas.
- Trabajé en modo de planeación, revisando cada paso antes de ejecutarlo.
- Pedí fuentes oficiales para cada afirmación relevante y revisé por mi cuenta las que el agente no pudo abrir.

2. Arquitectura actual y suposiciones



Cómo funciona hoy

Kova opera en **una sola cuenta de AWS** donde conviven producción y no productivo, sin separación formal y **sin infraestructura como código**: todo se creó y se modifica a mano en la consola.

- **Entrada de clientes:** los usuarios, en varios países, entran por AWS WAF y balanceadores de carga hacia decenas de servicios en ECS, repartidos en muchos clusters.
- **Entrada del equipo:** los ingenieros entran por una VPN y hacen los cambios directamente en la consola.
- **Datos y orquestación:** los servicios usan más de 20 bases en RDS, un dominio de OpenSearch y Temporal para los flujos de negocio.
- **Integraciones:** los servicios se conectan con proveedores de pagos locales y con la red pública Stellar, y firman transacciones con llaves de wallets.
- **Observabilidad:** la telemetría va a New Relic.

Los problemas principales:

- Un cambio de pruebas puede afectar a clientes.
- No hay forma de saber qué es producción ni quién cambió qué.
- No hay recuperación ante desastres.

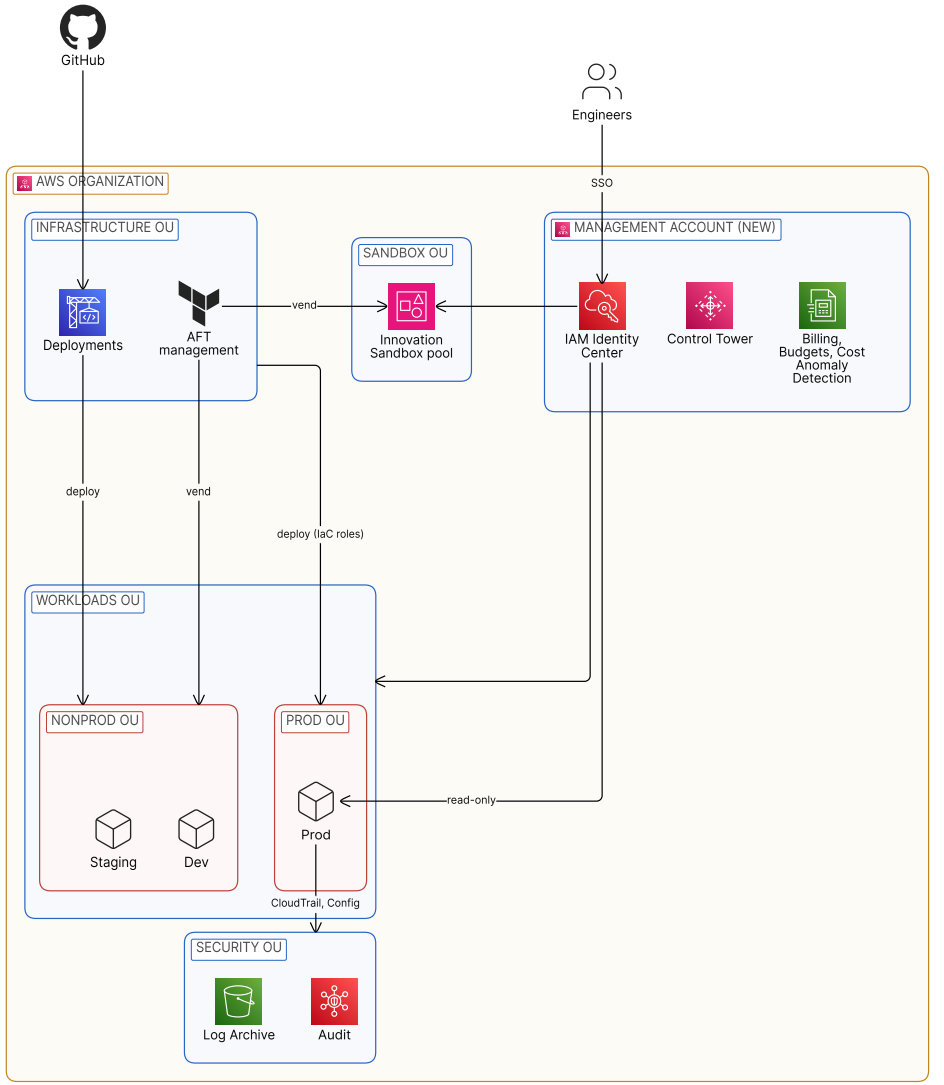
Suposiciones y ambigüedades del diagrama

Cada marca `?` del diagrama es una suposición o una ambigüedad. Todas están, junto con el resto, en la tabla de suposiciones.

Marca	Componente	Suposición
1?	Equipo	10 ingenieros con dos turnos de guardia 24/7 en varias zonas horarias.
2?	VPN	Producto y proveedor desconocidos; da acceso de red a la VPC y a los servicios de ECS.
3?	Balanceadores	Un ALB por servicio.
4?	ECS	Hasta 100 servicios, con un promedio de 2 vCPU y 4 GB por servicio; número de clusters, tipo de lanzamiento y autoescalado desconocidos (se asume autoescalado configurado).
5?	Temporal	Self-hosted en ECS, en la misma cuenta.
6?	OpenSearch	Dominio administrado de unos 100 GB; configuración y propósito (búsqueda o logs) desconocidos.
7?	RDS	Hasta 30 bases con motores, tamaños y propósitos diversos; en promedio, instancias grandes.
8?	Persistencia de Temporal	Se desconoce qué base y qué motor la respaldan; se asume una de las bases de RDS.
9?	Uso de KMS	Se desconoce cómo usan KMS los servicios (cifrado o firma).
10?	Llaves de firma de wallets	Guardadas en una base de RDS, no como llaves de KMS por wallet.
11?	Proveedores de pagos	Tienen en lista blanca las IPs de salida de Kova; esas direcciones no pueden cambiar.

Marca	Componente	Suposición
12?	Secretos de configuración	En Secrets Manager / SSM `SecureString` (práctica recomendada ya seguida).

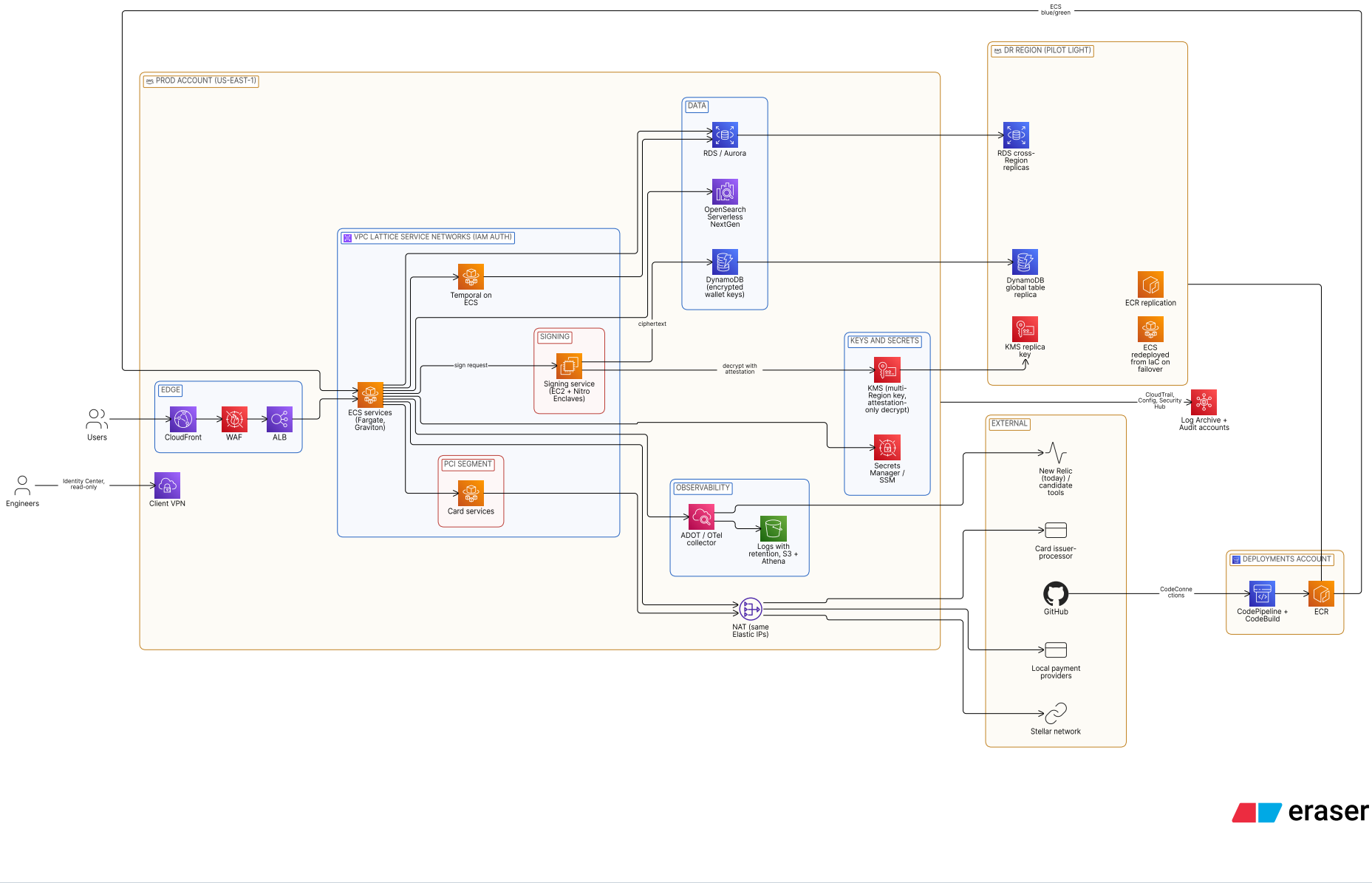
3. Arquitectura objetivo: organización y cuentas



eraser

- **O1** Una cuenta de administración nueva, sin cargas, con Organizations, Control Tower, IAM Identity Center y la facturación consolidada.
- **O2** Las cuentas Log Archive y Audit de Control Tower reciben CloudTrail, Config y Security Hub.
- **O3** AFT crea staging, dev y el pool de sandboxes, cada una con su línea base de roles, etiquetas y presupuestos.
- **O4** La cuenta actual se une sin cambios como producción, en una OU transitoria: personas solo lectura, cambios solo por IaC.
- **O5** Staging y dev reciben todo lo que no es producción; tienen SCPs restrictivas.
- **O6** Innovation Sandbox presta cuentas temporales con tope de costo y limpieza automática.
- **O7** CodePipeline y CodeBuild en la cuenta de Deployments despliegan en cada cuenta con roles de IaC.
- **O8** Los ingenieros entran con IAM Identity Center.

3. Arquitectura objetivo: plataforma de producción



1. Los usuarios entran por CloudFront y el WAF actual hacia los ALB; DNS y endpoints no cambian.
2. Los servicios de ECS pasan a Fargate con Graviton y rightsizing; Temporal sigue en ECS.
3. Los servicios se comunican por VPC Lattice con políticas de IAM.
4. Los servicios de tarjetas están en su propia service network; el issuer-processor guarda los datos de tarjeta.
5. Un servicio pide una firma al servicio de firma (EC2 con Nitro Enclaves).
6. El enclave lee la llave cifrada de DynamoDB, la descifra con una llave de KMS que solo acepta su atestación, firma y la descarta.
7. Stellar, los proveedores de pagos y el procesador se alcanzan por el mismo NAT y las mismas IPs de hoy.
8. Datos en RDS (o Aurora) y OpenSearch Serverless NextGen; secretos en Secrets Manager / SSM.
9. Todo servicio emite OpenTelemetry a un colector ADOT; New Relic se mantiene mientras se evalúan alternativas.
10. Todo log tiene retención; lo antiguo se archiva en S3 y se consulta con Athena.
11. GitHub, CodePipeline y CodeBuild, ECR y despliegue blue/green nativo de ECS.
12. El equipo entra por AWS Client VPN con IAM Identity Center; en producción solo lee.
13. CloudTrail, Config y Security Hub reportan a las cuentas Log Archive y Audit.
14. Región de DR en pilot light: réplicas de RDS, tabla global de DynamoDB, llave réplica de KMS y ECR replicado; ECS se crea con Terraform al conmutar.

Las decisiones detrás de cada paso están en decisiones_short.md.

4. Plan de migración y modernización

Cómo llevar a Kova de su estado actual a la arquitectura objetivo en 12 meses, sin downtime perceptible en un sistema financiero 24/7. El plan sigue las fases de la guía de AWS para migraciones grandes (evaluar, movilizar, migrar y modernizar) y las estrategias 7R. Las decisiones citadas están en decisiones_full.md.

La idea central

Producción no se mueve. La cuenta actual se queda como cuenta de producción (ADR-002); lo que sale es todo lo demás. Por eso la "migración" es en realidad una limpieza de la cuenta de producción más la construcción de la plataforma alrededor de ella:

Estrategia (7R)	Qué cubre
Retain	Producción: se queda en su cuenta y se adopta en IaC y en la organización.
Rehost / Relocate o rebuild	No productivo: sale a staging y dev, según la carga (ADR-012).
Retire	Recursos huérfanos encontrados en el inventario.
Replatform / Refactor	Modernización posterior: Fargate y Graviton, Aurora, OpenSearch Serverless NextGen, VPC Lattice, servicio de firma.

Principios de cero downtime

1. **Nada de producción se mueve** de cuenta, región, IP ni DNS.
2. **Todo cambio es aditivo y en paralelo**: lo nuevo se construye al lado de lo viejo y el tráfico se pasa de a poco.
3. **Todo cambio es reversible** y tiene un plan de vuelta atrás probado antes de empezar.
4. **Las herramientas de migración no tocan producción.** Se usan solo con el no productivo.
5. **Un import de Terraform solo se aplica si el plan no muestra cambios.**
6. **Despliegues blue/green** para servicios y bases de datos.
7. **Cortes en horas de menor tráfico** y congelamiento de cambios alrededor de eventos de alto volumen.

Qué hacemos primero y por qué

1. **Inventario y etiquetas.** Sin saber qué es producción y qué no, cualquier movimiento es una apuesta. Además resuelve las suposiciones `1?–12?`.
2. **La organización y la auditoría.** No cambian nada en producción y nos dan visibilidad (CloudTrail, Config) antes de tocar cualquier cosa.
3. **Adoptar producción en Terraform.** Una vez que producción está en código, cada cambio posterior es revisable y reversible.
4. **Sacar el no productivo.** Reduce el riesgo de que un cambio de pruebas afecte a clientes.
5. **Modernizar.** Solo cuando lo anterior está estable.

Qué no tocamos y por qué

Componente	Por qué no se toca
La cuenta de producción (no se cambia de cuenta)	Moverla es el mayor riesgo de downtime (ADR-002).
DNS, IPs públicas y de salida	Los proveedores de pagos tienen en lista blanca las IPs actuales (ADR-020).
Integración con la red Stellar	Pieza crítica sin contexto suficiente (ADR-032).
Temporal	El equipo lo conoce; migrarlo no aporta en 12 meses (ADR-016).
Llaves de KMS existentes	No se pueden mover entre cuentas (ADR-022).
CloudFront, WAF y ALB	Ingreso de clientes; solo se importan a IaC (ADR-018).
New Relic (durante la migración)	Es la visibilidad que necesitamos mientras cambiamos todo lo demás.

Fases

Fase 0. Evaluar (semanas 1 a 4)

- Activar AWS Config y Resource Explorer en la cuenta actual y sacar el inventario completo.
- Mapear dependencias con VPC Flow Logs y los mapas de servicios de New Relic.
- Clasificar cada recurso como producción, no productivo o huérfano, y etiquetar (`environment`, `service`, `owner`, `data-classification`).
- Responder las suposiciones: motores de RDS, persistencia de Temporal, dónde están las llaves de wallets, cómo es la VPN, qué balanceadores existen.
- Línea base de costos por servicio con las etiquetas nuevas y Cost Explorer.
- Abrir la conversación con el equipo de cuenta de AWS (ver Apoyo de AWS).

Criterio de salida: el 100 % de los recursos tiene `environment` y dueño; lista de huérfanos aprobada.

Fase 1. Movilizar: la fundación (semanas 3 a 10)

- Crear la cuenta de administración, la organización, Control Tower (Log Archive y Audit), IAM Identity Center y AFT (ADR-001, ADR-004).
- Revisar los acuerdos de facturación y luego invitar a la cuenta actual a la OU transitoria de producción, sin SCPs restrictivas. **Impacto en producción: ninguno.**
- CloudTrail organizacional, Config y Security Hub (ADR-030).
- Crear con AFT las cuentas Deployments, staging, dev y el pool de Innovation Sandbox, con SCPs restrictivas.
- Repositorio de Terraform, backend S3 y pipelines base (ADR-010, ADR-025).
- Permission sets de Identity Center: solo lectura para producción y rol *break-glass*. Los accesos actuales todavía no se quitan.

Criterio de salida: la cuenta de producción está en la organización, con CloudTrail centralizado y sin ningún incidente atribuible.

Fase 2. Adoptar producción en IaC (meses 2 a 5)

Se importa por componentes, de menor a mayor riesgo, con la regla de plan sin cambios (ADR-008):

1. Red: VPCs, subredes, security groups, NAT y Elastic IPs.
2. Ingreso: CloudFront, WAF, ALB, certificados y DNS.
3. Datos: RDS, OpenSearch, DynamoDB, KMS.
4. Cómputo: los servicios de ECS no se importan, se redefinen en Terraform y se redespliegan con blue/green, uno por uno.

En paralelo:

- Instrumentar con OpenTelemetry servicio por servicio, enviando a New Relic igual que hoy (ADR-027).
- Retención en todos los grupos de logs (ADR-028).
- Cuando el pipeline está probado para un equipo, se quitan sus permisos de escritura en producción (ADR-006).

Vuelta atrás: un import no cambia el recurso; si algo sale mal se quita del estado con `terraform state rm`. Un redespliegue blue/green vuelve solo al servicio anterior.

Criterio de salida: toda la producción está en Terraform y ninguna persona tiene permisos de escritura en producción.

Fase 3. Sacar el no productivo, por oleadas (meses 3 a 7)

Oleada	Contenido	Estrategia
1. Piloto	2 o 3 servicios de dev sin estado	Rebuild con Terraform en la cuenta dev
2. Dev	El resto de dev	Rebuild o rehost según la carga
3. Staging	Staging y sus bases	Rehost con snapshots compartidos, o seeds nuevos
4. Retiro	Recursos no productivos en la cuenta de producción	Detener, snapshot, cuarentena de 30 días y borrar

- Los datos de producción nunca se copian a no productivo sin anonimizar (Habeas Data y PCI).
- Cada oleada valida que producción no dependía de lo que se movió (los flujos de VPC y CloudTrail de la fase 0 lo muestran).

Vuelta atrás: hasta el fin de la cuarentena, los recursos viejos siguen en la cuenta de producción, detenidos y recuperables.

Fase 4. Plataforma objetivo alrededor de producción (meses 4 a 9)

Todo se construye en paralelo y se activa de a poco:

- **AWS Client VPN** en paralelo con la VPN actual; corte cuando todo el equipo migró (ADR-019).
- **VPC Lattice**: se crean las service networks y se mueven las llamadas entre servicios una por una; la ruta vieja queda hasta que la nueva está validada (ADR-017).
- **Segmentación PCI** de los servicios de tarjetas en su propia service network (ADR-033).

- **Servicio de firma con Nitro Enclaves** (ADR-021): 1. Ceremonia de llaves: cifrar las llaves existentes con la llave de KMS nueva y cargarlas en DynamoDB. 2. Modo sombra: el servicio nuevo firma en paralelo sin enviar; las firmas Ed25519 son deterministas, así que se comparan byte a byte con las actuales. 3. Corte por grupos de wallets, con vuelta atrás inmediata al firmante actual.
- **DR pilot light**: réplicas de RDS entre regiones, tabla global, llave réplica de KMS y replicación de ECR; primer simulacro de conmutación (ADR-031).
- **Evaluación de observabilidad**: el colector OpenTelemetry envía en paralelo a New Relic y a los candidatos.
- **FinOps**: presupuestos, anomalías y topes (ADR-029).

Fase 5. Modernizar (meses 6 a 12)

- **ECS** (ADR-013): Fargate y Graviton servicio por servicio con blue/green; rightsizing con las recomendaciones de AWS Compute Optimizer; Fargate Spot donde se tolere; consolidar clusters.
- **RDS** (ADR-014): RDS Blue/Green Deployments para cambios de versión y tamaño; migración a Aurora con réplica de lectura y promoción en horas de bajo tráfico.
- **OpenSearch** (ADR-015): doble escritura, carga histórica, cambio de lecturas por alias y el dominio viejo como vuelta atrás.
- **SCPs mínimas en producción** si el inventario muestra que son seguras (ADR-007).

Cronograma

Fase	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12
0. Evaluar	■											
1. Fundación	■	■	■									
2. Producción a IaC		■	■	■	■							
3. Sacar no productivo			■	■	■	■	■					
4. Plataforma objetivo				■	■	■	■	■	■			
5. Modernizar						■	■	■	■	■	■	■

Con un equipo de 10 personas y guardias 24/7, se asume que dos personas dedican la mayor parte de su tiempo a este plan y el resto participa por servicio.

Riesgos principales

Riesgo	Mitigación
Un recurso "de pruebas" lo usa producción	Mapa de dependencias de la fase 0 y cuarentena de 30 días antes de borrar.
Un import cambia un recurso de producción	Regla de plan sin cambios; revisión obligatoria del plan en el pipeline.
Quitar permisos humanos deja al equipo sin acción en un incidente	Rol *break-glass* probado antes de quitar permisos; pipeline de despliegue rápido.

Riesgo	Mitigación
Una firma incorrecta del servicio nuevo	Modo sombra con comparación exacta y corte por grupos de wallets.
El DR no cumple el RTO	Simulacros periódicos del `terraform apply` en la región de DR.
El equipo se satura	Fases secuenciales, oleadas pequeñas y apoyo de AWS.

Apoyo de AWS

- **Migration Acceleration Program (MAP)** sigue las mismas tres fases (evaluar, movilizar, migrar y modernizar) y puede aportar financiamiento y acompañamiento. Está pensado para migraciones hacia AWS; como Kova ya está en AWS, su aplicabilidad hay que confirmarla con el equipo de cuenta, sobre todo para la parte de modernización.
- **AWS Enterprise Support**: arquitecto técnico de cuenta, revisión Well-Architected y AWS Countdown para acompañar los cortes críticos.
- **AWS Professional Services o socios de AWS** para acelerar la fundación (Control Tower y AFT) o la adopción de IaC.
- **AWS Transform** para evaluaciones y modernización de código en etapas posteriores.

5. Decisiones técnicas y suposiciones

Resumen de las suposiciones y de las decisiones de arquitectura. El detalle de cada decisión (contexto, opciones, consecuencias y qué la cambiaría) está en decisiones_full.md. Los números `1?`–`12?` corresponden al diagrama de la arquitectura actual en arquitectura.md.

Suposiciones

ID	Tema	Suposición	Diagrama
S-01	Región	`us-east-1` como región principal; la región de DR no se define.	—
S-02	Tráfico	~10 mil solicitudes por minuto en promedio. Picos de 3 veces que duran una semana, cuatro veces al año (un mes de pico al año). El autoescalado probablemente ya está configurado.	`4?`
S-03	Cómputo	Hasta 100 servicios en ECS, con un promedio de 2 vCPU y 4 GB por servicio (cota superior), cada uno detrás de su propio ALB; número de clusters y tipo de lanzamiento desconocidos.	`3?` `4?`
S-04	Bases de datos	Hasta 30 instancias de RDS con motores y propósitos diversos. Para costos se promedian como instancias grandes en una sola zona (compensa bases medianas y bases Multi-AZ).	`7?`
S-05	OpenSearch	Dominio de unos 100 GB; configuración y propósito (búsqueda o logs) desconocidos.	`6?`
S-06	Temporal	Self-hosted en ECS en la misma cuenta; su persistencia es una de las bases de RDS.	`5?` `8?`
S-07	RTO / RPO	RTO de 1 hora para todo (por las dependencias entre sistemas) y RPO de 5 minutos; se espera una estrategia de DR.	—
S-08	Equipo	10 ingenieros, con dos turnos de guardia 24/7 en varias zonas horarias.	`1?`
S-09	Cumplimiento	Aplica PCI DSS (Kova emite tarjetas) a través de un issuer-processor; la Ley 1581 de Habeas Data ya se cumple.	—
S-10	Secretos y llaves	Secretos en Secrets Manager / SSM. Las llaves de firma de wallets se guardan hoy en una base de RDS, no como llaves de KMS por wallet.	`9?` `10?` `12?`
S-11	Red	DNS, IPs públicas e IPs de salida en listas blancas de proveedores no cambian.	`11?`
S-12	VPN	Producto y proveedor desconocidos; da acceso de red a las VPCs.	`2?`
S-13	Observabilidad	APM actual: New Relic; se evalúan alternativas sin preferencia previa.	—
S-14	Blockchain	Red Stellar (curva Ed25519); sin información de nodos ni de Horizon.	—
S-15	Código	El código vive en GitHub (o en su host actual).	—
S-16	Facturación	No hay acuerdos atados a la cuenta actual como pagadora que impidan unirla a una organización (se valida antes).	—

ID	Tema	Suposición	Diagrama
S-17	Wallets	El número de wallets es alto (miles o más), por lo que una llave de KMS por wallet no es viable en costo.	`10?`
S-18	Firmas	10 millones de firmas de transacciones al mes.	—
S-19	Base de costos	Solo precios bajo demanda en USD: sin Spot, sin Savings Plans ni instancias reservadas, sin capa gratuita. Para estimar, todo se trata como producción.	—

Decisiones

ID	Nombre	Descripción	Razón
ADR-001	Organización	Cuenta de administración nueva; la cuenta actual se invita como miembro	Práctica de AWS: sin cargas en la cuenta de administración, a la que no le aplican SCPs.
ADR-002	Cuenta de producción	La cuenta actual sigue siendo producción	Mover producción es el mayor riesgo de downtime; se conservan IDs, IPs, DNS y llaves.
ADR-003	OUs y cuentas	Security, Infrastructure, Workloads (Prod transitoria y NonProd) y Sandbox; cuentas por entorno	Simple de operar para un equipo de 10.
ADR-004	Creación de cuentas	Control Tower con AFT	El requisito es Terraform; LZA es CDK.
ADR-005	Sandboxes	Innovation Sandbox on AWS; se acepta el drift de Control Tower como señal	Presupuesto y limpieza automática por cuenta.
ADR-006	Acceso a producción	Personas solo lectura; cambios solo por roles de IaC; *break-glass* alertado	La IaC como única vía de cambio; mínimo privilegio.
ADR-007	SCPs	Restrictivas en todas las cuentas menos producción	No romper flujos de producción no documentados.
ADR-008	Adopción de IaC	Importar lo sensible y expuesto; reconstruir lo efímero	Adoptar producción sin tocarla.
ADR-009	Herramienta de IaC	Terraform con `aws` y `awscc`	Requisito; `awscc` cubre servicios nuevos.
ADR-010	Layout de Terraform	Patrones `aws-ia` / `aws-samples`; S3 con bloqueo nativo; un estado por cuenta y componente	Estándar de la industria; estados pequeños.
ADR-011	Estrategia de migración	Retain para producción, sacar el no productivo, retire de huérfanos	Cero downtime: producción no se mueve.
ADR-012	No productivo	Rehost o rebuild según la carga (abierto)	No bloquear la limpieza en casos difíciles.
ADR-013	Cómputo	Seguir en ECS; Fargate, Graviton, rightsizing, Spot	El equipo conoce ECS; menos operación y costo.
ADR-014	Bases de datos	Rightsizing o Aurora según motor y propósito, con Blue/Green	Ahorro sin downtime perceptible.
ADR-015	OpenSearch	OpenSearch Serverless NextGen	Escala a cero y cobra por uso.

ID	Nombre	Descripción	Razón
ADR-016	Orquestación	Temporal self-hosted en ECS, sin cambios por 12 meses	Experiencia del equipo; otra migración no aporta.
ADR-017	Conectividad entre servicios	VPC Lattice con autenticación de IAM	Confianza cero, sin problemas de CIDR; costo aceptado.
ADR-018	Ingreso	CloudFront, WAF y ALB sin cambios	Cero cambio visible para los clientes.
ADR-019	Acceso del equipo	AWS Client VPN con IAM Identity Center	Una sola identidad, auditada.
ADR-020	Direcciones	DNS, IPs y NAT sin cambios	Listas blancas de proveedores intactas.
ADR-021	Llaves de wallets	Cifrado de sobre: DynamoDB + KMS con atestación + Nitro Enclaves en EC2	Costo independiente del número de wallets; la llave en claro solo existe en el enclave. CloudHSM descartado por costo.
ADR-022	Ubicación de las llaves	Se quedan en producción	Las llaves de KMS no se mueven entre cuentas.
ADR-023	Origen de las llaves	Conservar, importar o generar fuera e importar	Portabilidad, sin atarse a un proveedor.
ADR-024	Secretos	Secrets Manager y SSM `SecureString`	Práctica recomendada, ya en uso.
ADR-025	CI/CD	CodePipeline y CodeBuild en una cuenta de Deployments; GitHub vía CodeConnections; pipelines de Terraform, de aplicaciones, de la imagen del enclave y de AFT	Todo en AWS y sin credenciales permanentes; riesgo: la cuenta de Deployments puede cambiar producción, mitigado con SCPs, roles separados de plan y apply, y aprobaciones
ADR-026	Estrategia de despliegue	Blue/green nativo de ECS, sin CodeDeploy	Menos piezas.
ADR-027	Observabilidad	OpenTelemetry en todo; New Relic se mantiene; se evalúan alternativas	Cambiar de herramienta sin re-instrumentar.
ADR-028	Logs	Retención en todo; archivo en S3 consultado con Athena	Costo predecible.
ADR-029	FinOps	Etiquetas obligatorias, Budgets, Cost Anomaly Detection, topes en dev y sandboxes	Saber qué cuesta cada cosa.
ADR-030	Auditoría	CloudTrail organizacional, Config (paquete PCI) y Security Hub	Trazabilidad y hallazgos centralizados.
ADR-031	DR	Pilot light multirregión; DRS descartado	Cumple RTO/RPO al menor costo; DRS no cubre ECS ni RDS.
ADR-032	Blockchain	No se toca la integración con Stellar	Sin contexto suficiente; pieza crítica.
ADR-033	PCI	Issuer-processor + segmentación con Lattice dentro de producción	Alcance mínimo sin mover nada.

6. Estimado de costos y recortes

- **Calculadora de AWS:** *enlaces pendientes*. Las líneas de costos.xlsx y costos-actual.xlsx sirven de especificación para cargarlas.
- **Archivos:** cada libro tiene una hoja `Supuestos` (todas las entradas) y una hoja `Estimado` (líneas con fórmulas). El del objetivo tiene además una hoja `Recortes`.
- **Servicios fuera de la calculadora:** OpenSearch Serverless NextGen y CloudWatch Omni se calculan solo en el archivo, con los precios de su página.

Base del estimado

- Región us-east-1, en USD.
- Solo precios bajo demanda: sin Spot, sin Savings Plans ni instancias reservadas, sin capa gratuita.
- Todo se trata como producción (S-19).
- **Cómputo:** 100 servicios de ECS con 2 vCPU y 4 GB cada uno, como cota superior.
- **Balanceadores:** un ALB por servicio.
- **Bases:** 30 instancias grandes de RDS PostgreSQL en una zona.
- **OpenSearch:** 100 GB.
- **Firmas:** 10 millones al mes, con dos `c9g.large` para el servicio de firma.
- **Picos:** de 3 veces, una semana cuatro veces al año, es decir un mes de pico al año. El cómputo y el tráfico usan un factor promedio de (11 + 3) / 12 ≈ 1.17.
- **Precios unitarios:** son los de la página de precios de cada servicio y se confirman al cargar la calculadora.

Costo mensual: actual frente a objetivo

Categoría	Actual (USD / mes)	Objetivo (USD / mes)	Diferencia
Cómputo	8,409	6,845	−1,564
Red	2,628	5,444	+2,816
Datos	12,298	11,905	−393
Llaves y secretos	130	142	+12
Observabilidad	1,505	1,175	−330
Gobierno	0	260	+260
CI/CD	0	246	+246
DR	0	11,262	+11,262
Sandboxes	0	500	+500
Total mensual	**~24,970**	**~37,780**	**+12,810 (+51 %)**
Total anual	**~299,650**	**~453,340**	

Lo que agrega el objetivo:

- El DR en pilot light, que hoy no existe: casi toda la diferencia es el costo de tener las 30 bases replicadas en otra región.
- VPC Lattice, CloudFront y AWS Client VPN.
- El gobierno (Config, Security Hub), los pipelines y los sandboxes.

Lo que ahorra:

- Fargate en Graviton.
- La retención de logs con archivo en S3.
- OpenSearch Serverless en lugar de un dominio con nodos fijos.

Sin DR, el objetivo costaría casi lo mismo que hoy, con mucho más control.

No incluye: licencias de New Relic u otra herramienta de APM, la licencia de la VPN actual, el plan de soporte de AWS, impuestos ni el costo del equipo.

Dos recortes concretos y el riesgo que agregan

Recorte	Ahorro (USD / mes)	Riesgo que agrega
1. DR solo con copias de AWS Backup entre regiones, sin réplicas vivas de RDS (ADR-031)	~9,570	El RPO pasa de minutos a horas (la frecuencia de las copias) y el RTO crece porque hay que restaurar 30 bases antes de levantar los servicios; el RTO de 1 hora deja de cumplirse.
2. Reemplazar VPC Lattice por ECS Service Connect (ADR-017)	~1,740	Se pierde la autorización de IAM entre servicios: la segmentación del servicio de firma y de los servicios de tarjetas (PCI) queda solo en security groups, y la comunicación entre VPCs o cuentas vuelve a depender del enrutamiento de red.

Otros candidatos evaluados, en la hoja `Recortes`:

Candidato	Ahorro (USD / mes)	Riesgo
Consolidar los 100 ALB en unos 10	~1,480	Mayor radio de impacto si falla un ALB compartido.
Fargate Spot para el 30 % de los servicios	~1,410	Interrupciones con dos minutos de aviso.
Compute Savings Plan a un año	~1,370	Compromiso de gasto aunque el uso baje.

7. Controles de seguridad de los primeros 90 días

Principios: nadie cambia producción a mano, todo queda registrado en CloudTrail, y personas y cargas tienen el mínimo privilegio. Las SCPs gobiernan todas las cuentas de la organización salvo producción, que se controla con IAM mientras se inventaría (ADR-007).

Área	Días 0 a 30	Días 31 a 60	Días 61 a 90
Accesos	IAM Identity Center como única entrada; MFA obligatorio; rol *break-glass* con alerta; inventario de usuarios IAM y llaves de acceso.	Permission sets de solo lectura en producción; quitar usuarios IAM y llaves de acceso de larga duración.	Sin permisos de escritura humanos en producción; roles de cargas revisados con IAM Access Analyzer (permisos no usados).
Secretos y llaves	Inventario de secretos y de dónde están las llaves de wallets.	Todo secreto en Secrets Manager / SSM con llave de KMS por cuenta; rotación donde aplique.	Política de la llave de wallets limitada a la atestación del enclave (ADR-021).
Red	Revisar security groups abiertos a internet y reglas del WAF.	AWS Client VPN con Identity Center en paralelo a la VPN actual.	Servicios sensibles (tarjetas, firma) detrás de VPC Lattice con políticas de IAM.
Auditoría	CloudTrail organizacional hacia Log Archive; AWS Config en todas las cuentas.	Security Hub en la cuenta Audit; paquete de conformidad PCI DSS de Config.	Alertas sobre acciones críticas: uso de *break-glass*, cambios de IAM o de políticas de llaves, cambios fuera del pipeline.
Respuesta a incidentes	Guardia 24/7 con dos turnos y canal de escalamiento definido.	Runbooks para credenciales filtradas, llave comprometida y cambio no autorizado.	Simulacro de incidente y revisión posterior.

Mínimo privilegio

- **Personas:** permission sets por función (lectura, operación en dev, administración de plataforma) asignados por grupo, nunca por usuario. En producción solo lectura; el rol *break-glass* requiere MFA, deja alerta y se revisa después de cada uso.
- **Cargas:** un rol por servicio de ECS con solo las acciones y recursos que usa. El servicio de firma es el único que puede pedir el descifrado de las llaves de wallets, y solo desde el enclave.
- **Pipelines:** los roles de IaC se asumen solo desde la cuenta de Deployments, sin credenciales permanentes.

SCPs en las cuentas que no son producción

- Solo regiones aprobadas.
- Prohibido desactivar CloudTrail, Config o Security Hub, o salir de la organización.
- Prohibido crear usuarios IAM o llaves de acceso.
- Recursos sin etiquetas obligatorias rechazados.
- Topes de costo en sandboxes (Innovation Sandbox) y dev (Budgets).

8. Infraestructura como código

Esqueleto de Terraform de la arquitectura objetivo. No está pensado para desplegarse tal cual: muestra la estructura, las convenciones y ejemplos concretos de las decisiones de decisiones_full.md. Todos los componentes pasan `terraform fmt` y `terraform validate`.

Estructura

Basada en los patrones de `aws-ia` y `aws-samples` (ADR-010): módulos reutilizables en `modules/` y componentes desplegables en `live/<cuenta>/<componente>`, cada uno con su propio estado.

```
infra/
├── modules/
│   ├── ecs-service/          # Ejemplo principal: servicio ECS de la plataforma objetivo
│   ├── signing-enclave/      # Servicio de firma: Nitro Enclaves + KMS + DynamoDB global
│   └── search-collection/    # OpenSearch Serverless con el proveedor awscc
└── live/
    ├── management/
    │   └── organization/     # Organización, OUs, SCPs, políticas de etiquetas, Control Tower
    ├── aft/                  # Account Factory for Terraform
    └── prod/                 # Cuenta actual, adoptada en IaC
        ├── network/          # VPC, NAT y Elastic IP importados + service networks de Lattice
        ├── data/             # RDS importado + réplica en la región de DR
        ├── services/
        │   └── payments-api/ # Uso del módulo ecs-service
        ├── signing/          # Uso del módulo signing-enclave
        └── search/           # Uso del módulo search-collection
```

Las cuentas staging, dev, Deployments y el pool de sandboxes se piden a AFT desde su propio repositorio (`aft-account-request`), un archivo por cuenta.

Convenciones

- **Proveedores:** `aws` por defecto y `awscc` para capacidades nuevas, como OpenSearch Serverless (ADR-009).
- **Estado:** backend S3 con bloqueo nativo (`use_lockfile = true`), un estado por cuenta y componente. Los componentes comparten identificadores por SSM Parameter Store, no leyendo el estado de otros.
- **Acceso:** el pipeline asume un rol de IaC en cada cuenta (`deploy_role_arn`); ninguna persona aplica cambios en producción (ADR-006).
- **Etiquetas:** `default_tags` del proveedor con `environment`, `component`, `owner` y `managed-by` (ADR-029).
- **Adopción de producción:** bloques `import` y `prevent_destroy` para lo sensible; un import solo se aplica con un plan sin cambios (ADR-008).

Ejemplo principal: `modules/ecs-service`

Un servicio de ECS con las decisiones de la plataforma aplicadas:

- Fargate sobre Graviton (`ARM64`) y una proporción opcional en Fargate Spot (ADR-013).

- Despliegue blue/green nativo de ECS con *circuit breaker* y *rollback* (ADR-026).
- Registro en VPC Lattice para el tráfico entre servicios (ADR-017).
- Colector ADOT como *sidecar* para OpenTelemetry (ADR-027).
- Grupo de logs con retención (ADR-028).
- Rol de tarea propio con una política de mínimo privilegio, y rol de ejecución que solo lee los secretos del servicio.
- Autoescalado por CPU.

`live/prod/services/payments-api` muestra cómo se usa.

Validar

```
cd infra/live/prod/services/payments-api
```

```
terraform init -backend=false
```

```
terraform validate
```

Anexo A. Decisiones en detalle

Cada decisión sigue el formato ADR: contexto, opciones, decisión, consecuencias y qué cambiaría la decisión. Las suposiciones en las que se apoyan están en decisiones_short.md. Los números de paso (`01–08` y `1–14`) hacen referencia a los diagramas de arquitectura.md.

Tabla de Contenidos

Organización y gobierno

1. ADR-001: Organización de AWS con una cuenta de administración nueva
2. ADR-002: La cuenta actual se mantiene como cuenta de producción
3. ADR-003: Estructura de OUs y cuentas
4. ADR-004: AFT sobre Control Tower para crear cuentas
5. ADR-005: Innovation Sandbox on AWS para los sandboxes
6. ADR-006: Acceso humano de solo lectura en producción
7. ADR-007: SCPs en todas las cuentas excepto producción

Infraestructura como código

8. ADR-008: Todo a IaC: importar lo sensible, reconstruir lo efímero
9. ADR-009: Terraform con los proveedores aws y awscc
10. ADR-010: Estructura del repositorio y estado de Terraform

Migración y modernización

11. ADR-011: La migración es una limpieza de la cuenta de producción
12. ADR-012: Rehost o rebuild del no productivo según la carga
13. ADR-013: Modernización del cómputo en ECS
14. ADR-014: Modernización de RDS según motor y propósito
15. ADR-015: OpenSearch Serverless NextGen
16. ADR-016: Temporal se mantiene self-hosted durante 12 meses

Red y acceso

17. ADR-017: VPC Lattice en lugar de Transit Gateway
18. ADR-018: Ingreso con CloudFront, WAF y ALB sin cambios
19. ADR-019: AWS Client VPN e IAM Identity Center para el equipo
20. ADR-020: DNS, IPs y direcciones de salida sin cambios

Llaves y secretos

21. ADR-021: Llaves de firma de wallets con Nitro Enclaves, KMS y DynamoDB
22. ADR-022: Las llaves permanecen en la cuenta de producción
23. ADR-023: Origen de las llaves: conservar, importar o generar fuera e importar
24. ADR-024: Secretos de configuración en Secrets Manager y SSM

CI/CD

25. ADR-025: CodePipeline y CodeBuild en una cuenta de Deployments
26. ADR-026: Despliegues blue/green nativos de ECS

Observabilidad, FinOps y seguridad

27. ADR-027: OpenTelemetry como capa de desacople de observabilidad
28. ADR-028: Retención de logs y archivo en S3 con Athena
29. ADR-029: FinOps: etiquetado obligatorio, presupuestos y anomalías
30. ADR-030: CloudTrail organizacional, AWS Config y Security Hub

Resiliencia, blockchain y cumplimiento

31. ADR-031: DR pilot light en una segunda región, sin DRS
32. ADR-032: No tocar la infraestructura blockchain
33. ADR-033: Reducción del alcance PCI con issuer-processor y segmentación

---

ADR-001: Organización de AWS con una cuenta de administración nueva

Contexto

Hoy hay una sola cuenta donde conviven producción y no productivo. Se necesita separar entornos, gobernar con políticas y consolidar la facturación. La cuenta que crea la organización (cuenta de administración) tiene privilegios especiales y las SCPs no le aplican.

Opciones consideradas

- Crear la organización desde la cuenta actual de producción.
- Crear una cuenta nueva solo para administrar la organización e invitar a la cuenta actual.
- Seguir con una sola cuenta y separar entornos con etiquetas y roles.

Decisión

Se crea una cuenta de administración nueva, sin cargas de trabajo, con AWS Organizations, Control Tower, IAM Identity Center y la facturación consolidada (paso `01`). La cuenta actual se invita como miembro.

Consecuencias

Positivas

- Sigue la práctica recomendada por AWS de no desplegar cargas en la cuenta de administración.
- Un compromiso de producción no compromete la organización completa.
- Unirse a una organización no cambia el ID de la cuenta, y las RIs y Savings Plans se quedan con ella.

Negativas

- La cuenta nueva pasa a ser la pagadora: hay que revisar antes de invitar a producción los acuerdos atados al pagador (EDP o precios privados, ofertas privadas de Marketplace, plan de soporte).
- Una cuenta más que administrar.

Qué cambiaría la decisión

- Que exista un acuerdo comercial atado a la cuenta actual como pagadora que no se pueda transferir.

ADR-002: La cuenta actual se mantiene como cuenta de producción

Contexto

Crear una cuenta de producción nueva implicaría mover más de 100 servicios, hasta 30 bases de datos, llaves de KMS (que no se pueden mover entre cuentas), IPs registradas en proveedores y DNS, en un sistema financiero 24/7.

Opciones consideradas

- Nueva cuenta de producción y migrar todo.
- Mantener la cuenta actual como producción y sacar todo lo demás.

Decisión

La cuenta actual se queda como producción (paso `04`). Lo que sale es todo lo que no es producción (ver ADR-011).

Consecuencias

Positivas

- Producción no se mueve: es el principal argumento de cero downtime.
- Se conservan IDs, IPs, DNS, llaves de KMS, cuotas y descuentos.

Negativas

- La cuenta arrastra deuda histórica (recursos huérfanos, permisos amplios) que hay que limpiar en sitio.
- No se obtiene una cuenta "limpia" de producción desde el día uno.

Qué cambiaría la decisión

- Que la cuenta actual estuviera comprometida o que la deuda fuera tan grande que limpiar en sitio fuera más riesgoso que mover.

ADR-003: Estructura de OUs y cuentas

Contexto

Se necesita una estructura simple que un equipo de 10 ingenieros pueda operar, alineada con las guías de AWS para entornos multicuenta.

Opciones consideradas

- Cuentas por entorno (prod, staging, dev).
- Cuentas por entorno y por dominio (pagos, wallets, tarjetas).

Decisión

Cuentas por entorno, con estas OUs (diagrama de organización):

OU	Cuentas
Security	Log Archive, Audit (las crea Control Tower)
Infrastructure	AFT management, Deployments
Workloads / Prod (transitoria)	Producción (cuenta actual)
Workloads / NonProd	Staging, Dev
Sandbox	Pool de Innovation Sandbox

Consecuencias

Positivas

- Pocas cuentas, fáciles de operar con un equipo pequeño.
- Separa entornos y reduce el radio de impacto entre ellos.

Negativas

- Dentro de producción no hay separación por dominio a nivel de cuenta; se compensa con VPC Lattice, IAM y segmentación (ADR-017, ADR-033).

Qué cambiaría la decisión

- Crecimiento del equipo a varios equipos autónomos por dominio, o un requisito regulatorio de aislar un dominio por cuenta.

ADR-004: AFT sobre Control Tower para crear cuentas

Contexto

El requisito de IaC es Terraform. Las opciones de AWS para crear y personalizar cuentas son Account Factory for Terraform (AFT) y Landing Zone Accelerator (LZA).

Opciones consideradas

- LZA: más controles de cumplimiento, pero está construido en CDK y se configura con YAML propio.
- AFT: nativo de Terraform, sobre Control Tower.
- Organizations sin Control Tower, todo a mano en Terraform.

Decisión

Control Tower con AFT (paso `03`). Se descarta LZA porque es CDK y obligaría a mantener dos herramientas de IaC.

Consecuencias

Positivas

- Una sola herramienta de IaC para la plataforma y las cargas.
- Cuentas nuevas con línea base (roles, presupuestos, etiquetas) desde el primer minuto.

Negativas

- AFT trae menos controles de cumplimiento listos que LZA; se completan con Config y Security Hub (ADR-030).
- AFT agrega su propia cuenta y su propio pipeline que operar.

Qué cambiaría la decisión

- Que el requisito de Terraform se relajara y el peso regulatorio justificara LZA.

ADR-005: Innovation Sandbox on AWS para los sandboxes

Contexto

Los ingenieros necesitan experimentar sin tocar producción ni acumular costos olvidados.

Opciones consideradas

- Una cuenta sandbox compartida.
- Innovation Sandbox on AWS con un pool de cuentas temporales.

Decisión

Innovation Sandbox on AWS con un pool de cuentas pequeñas y limitadas creadas por AFT (paso `06`). Innovation Sandbox mueve cuentas entre OUs y Control Tower lo reporta como drift; se acepta y se usa como señal de monitoreo.

Consecuencias

Positivas

- Presupuesto y limpieza automática por cuenta prestada.
- Los experimentos no ensucian dev ni producción.

Negativas

- Alertas de drift en Control Tower que hay que saber filtrar.
- La solución no crea cuentas; el pool se crea y se mantiene aparte.

Qué cambiaría la decisión

- Que el costo o la operación del pool supere su uso real; se volvería a una cuenta sandbox con presupuesto.

ADR-006: Acceso humano de solo lectura en producción

Contexto

Hoy los cambios se hacen a mano en la consola. Para que la IaC sea la fuente de verdad, nadie debe poder cambiar producción fuera del pipeline.

Opciones consideradas

- Acceso de administrador con aprobación manual.
- Solo lectura para personas; cambios solo por roles de IaC; rol de emergencia.

Decisión

En producción, todas las personas tienen solo lectura (vía IAM Identity Center). Solo los roles que asume el pipeline de Terraform y de despliegue pueden mutar recursos. Existe un rol *break-glass* con MFA, alerta inmediata y revisión posterior. Todo queda en CloudTrail.

Consecuencias

Positivas

- La consola deja de ser una vía de cambio; el drift se vuelve excepción.
- Mínimo privilegio para humanos y cargas.

Negativas

- Las emergencias dependen del rol *break-glass* y de un pipeline rápido.
- Requiere disciplina y un pipeline confiable antes de quitar permisos.

Qué cambiaría la decisión

- Nada a mediano plazo; solo se ajustaría qué lecturas se permiten (por ejemplo, ocultar datos sensibles en consola).

ADR-007: SCPs en todas las cuentas excepto producción

Contexto

Las SCPs limitan lo que puede hacer cualquier identidad de una cuenta. Aplicarlas de golpe a una cuenta de producción construida a mano puede romper flujos que nadie documentó.

Opciones consideradas

- SCPs restrictivas en todas las cuentas, incluida producción.
- SCPs restrictivas en las cuentas nuevas; producción en una OU transitoria sin SCPs restrictivas.

Decisión

Las cuentas nuevas (staging, dev, sandboxes, seguridad, infraestructura) tienen SCPs restrictivas desde el inicio (regiones permitidas, no desactivar CloudTrail/Config, no salir de la organización, etc.). Producción vive en una OU transitoria sin SCPs restrictivas; su control principal es IAM (ADR-006).

Consecuencias

Positivas

- Cero riesgo de que una SCP corte un flujo de producción no documentado.
- Las cuentas nuevas nacen gobernadas.

Negativas

- Producción depende de IAM y de la disciplina del pipeline, sin la red de seguridad de las SCPs.

Qué cambiaría la decisión

- Una vez inventariados los flujos de producción, se podrían agregar SCPs mínimas de bajo riesgo (no desactivar CloudTrail, no salir de la organización) usando el análisis de *last accessed* de IAM.

ADR-008: Todo a IaC: importar lo sensible, reconstruir lo efímero

Contexto

Toda la infraestructura se creó a mano. Recrear recursos con estado o expuestos a clientes es riesgoso; recrear recursos sin estado es barato.

Opciones consideradas

- Importar todo al estado de Terraform.
- Reconstruir todo desde cero.
- Mezcla según el tipo de recurso.

Decisión

Se **importa** con bloques `import` todo lo que da la cara al cliente o es sensible: VPCs, ALBs, WAF, DNS, RDS, OpenSearch, KMS, DynamoDB, certificados. Se **reconstruye** lo efímero y sin estado: servicios y task definitions de ECS, configuración de clusters. La regla de importación es "plan sin cambios": un import solo se aplica si `terraform plan` no muestra diferencias.

Consecuencias

Positivas

- Se adopta producción sin tocarla.
- Lo reconstruido queda limpio y estandarizado desde el inicio.

Negativas

- Importar es lento y manual; hay que generar y revisar la configuración de cientos de recursos (se puede acelerar con `terraform plan -generate-config-out` y asistentes de IA).

Qué cambiaría la decisión

- Si un recurso importado no se puede describir sin cambios, se reemplaza con una estrategia blue/green en lugar de forzarlo.

ADR-009: Terraform con los proveedores aws y awscc

Contexto

El requisito es Terraform. Algunos servicios nuevos o capacidades poco comunes aparecen antes en el proveedor `awscc` (generado desde la API de Cloud Control) que en el proveedor `aws`.

Decisión

Terraform con el proveedor `aws` por defecto y `awscc` para servicios o capacidades que aún no estén en `aws` (por ejemplo, opciones nuevas de OpenSearch Serverless NextGen).

Consecuencias

Positivas

- Acceso temprano a capacidades nuevas sin salir de Terraform.

Negativas

- Dos proveedores con convenciones distintas; hay que documentar cuándo usar cada uno.

Qué cambiaría la decisión

- Que todo lo necesario esté en el proveedor `aws`; `awscc` quedaría sin uso.

ADR-010: Estructura del repositorio y estado de Terraform

Contexto

No hay experiencia previa en el equipo con un layout de Terraform. Conviene seguir patrones conocidos.

Decisión

- Layout basado en los patrones de `aws-ia` y `aws-samples`: `modules/` reutilizables y `live/<cuenta>/<componente>` desplegables.
- Backend S3 con bloqueo nativo (`use_lockfile`), un estado por cuenta y componente.
- Ver el README de infra.

Consecuencias

Positivas

- Estados pequeños: un `plan` afecta poco y es rápido.
- Patrones reconocibles por cualquier ingeniero nuevo.

Negativas

- Más estados que coordinar; las dependencias entre componentes se resuelven con datos remotos o SSM.

Qué cambiaría la decisión

- Que el número de componentes crezca tanto que convenga una herramienta de orquestación (por ejemplo, Terragrunt o stacks de Terraform).

ADR-011: La migración es una limpieza de la cuenta de producción

Contexto

La guía de AWS para migraciones grandes clasifica cada carga con las 7R (retire, retain, rehost, relocate, repurchase, replatform, refactor). Aquí todo ya está en AWS; el problema es que producción y no productivo están mezclados.

Decisión

- Producción: **retain**. Se queda en su cuenta y se adopta en IaC y gobierno.
- No productivo: sale a staging y dev (ADR-012).
- Lo huérfano: **retire**.
- La modernización (replatform, refactor) viene después, servicio por servicio (ADR-013 a ADR-016).

Las herramientas de migración de servidores (MGN, AWS Transform) aplican a modernizaciones futuras y al no productivo donde haya servidores; producción son contenedores y datos administrados.

Consecuencias

Positivas

- Cero downtime: lo que se mueve no es producción.
- Las herramientas de migración no tocan la infraestructura de producción.

Negativas

- Hay que separar con precisión qué es producción y qué no en una cuenta sin etiquetas; es el riesgo principal de la fase de inventario.

Qué cambiaría la decisión

- Encontrar dependencias de producción hacia recursos no productivos (por ejemplo, un servicio productivo que lee una base "de pruebas"); ese recurso se reclasifica como producción.

ADR-012: Rehost o rebuild del no productivo según la carga

Contexto

El no productivo puede moverse tal cual (rehost/relocate) o reconstruirse desde IaC (rebuild).

Decisión

Se decide por carga y se deja abierto a propósito. Criterio guía: si ya existe la definición en IaC o es sin estado, se reconstruye; si tiene estado o configuración difícil de reproducir, se mueve (snapshots compartidos, réplicas).

Consecuencias

Positivas

- Flexibilidad para no bloquear la limpieza en las cargas difíciles.

Negativas

- Sin una regla única, hay que justificar cada caso.

Qué cambiaría la decisión

- Un inventario que muestre que casi todo es reconstruible; entonces se reconstruye todo.

ADR-013: Modernización del cómputo en ECS

Contexto

Hay hasta 100 servicios en muchos clusters, sin datos de uso. El equipo ya conoce ECS.

Opciones consideradas

- Migrar a EKS.
- Quedarse en ECS y optimizar.

Decisión

Se queda en ECS (paso `2`). Después de la migración:

- Reorganizar clusters por dominio y entorno.
- Fargate y Graviton en todo lo posible; rightsizing con métricas reales.
- Fargate Spot donde la carga tolere interrupciones.
- A largo plazo y por carga: dividir monolitos, o pasar APIs simples a Lambda y API Gateway.

Consecuencias

Positivas

- Menos operación (sin nodos que parchar) y menor costo por Graviton y rightsizing.
- Sin curva de aprendizaje de Kubernetes.

Negativas

- Algunas imágenes tendrán que compilarse para ARM64.

Qué cambiaría la decisión

- Necesidades que solo cubra Kubernetes (operadores específicos, portabilidad multinube).

ADR-014: Modernización de RDS según motor y propósito

Contexto

Hasta 30 bases con motores, tamaños y configuraciones diversos y sin datos de uso.

Decisión

Se evalúa base por base: rightsizing y autoscaling de almacenamiento, o migración a Aurora cuando el motor lo permite y el uso lo justifica. Los cambios se hacen con RDS Blue/Green Deployments o promoviendo réplicas de lectura de Aurora, para cortes de segundos.

Consecuencias

Positivas

- Ahorro y elasticidad sin downtime perceptible.

Negativas

- Cada base requiere pruebas propias; es la parte más lenta del plan.

Qué cambiaría la decisión

- Motores sin camino a Aurora; se quedan en RDS con rightsizing.

ADR-015: OpenSearch Serverless NextGen

Contexto

Hay dominios de OpenSearch administrados con configuración desconocida. La nueva generación de OpenSearch Serverless (GA en mayo de 2026) escala hasta cero y cobra por uso.

Decisión

Migrar a OpenSearch Serverless NextGen como parte de la modernización, con doble escritura y cambio de alias para no tener downtime.

Consecuencias

Positivas

- Sin clusters que dimensionar; costo alineado con el uso.

Negativas

- Servicio nuevo: hay que validar compatibilidad de funcionalidades y que esté en la calculadora de precios (si no, se estima aparte).

Qué cambiaría la decisión

- Funcionalidades que solo existan en dominios administrados, o un uso constante tan alto que el aprovisionado sea más barato.

ADR-016: Temporal se mantiene self-hosted durante 12 meses

Contexto

Temporal orquesta los flujos de negocio y se asume self-hosted en ECS. El equipo lo conoce.

Opciones consideradas

- Temporal Cloud.
- Reescribir flujos en Step Functions, MWAA o Glue.
- Mantenerlo como está.

Decisión

Se mantiene self-hosted en ECS (paso `2`) durante los 12 meses. No se migra a Temporal Cloud (sería otra migración) ni se reescriben los flujos. A futuro se pueden mover flujos puntuales a Step Functions con EventBridge, o a Glue, si hay un caso de costo de largo plazo.

Consecuencias

Positivas

- Se aprovecha la experiencia del equipo y no se agrega riesgo.

Negativas

- Se sigue operando Temporal y su persistencia.

Qué cambiaría la decisión

- Que la operación de Temporal consuma una parte relevante del tiempo del equipo.

ADR-017: VPC Lattice en lugar de Transit Gateway

Contexto

Al separar entornos y segmentar servicios sensibles, los servicios necesitan conectarse entre VPCs y cuentas con control de acceso.

Opciones consideradas

- Transit Gateway con reglas de red.
- VPC peering.
- VPC Lattice.

Decisión

VPC Lattice para la comunicación entre servicios (paso `3`): service networks con políticas de autenticación de IAM, integrado de forma nativa con ECS.

Consecuencias

Positivas

- Modelo de confianza cero entre servicios con IAM, sin administrar TLS en cada ALB interno.
- No importan los CIDR superpuestos entre VPCs.
- Útil para segmentar los servicios de tarjetas y el servicio de firma.

Negativas

- Más costoso que TGW o peering a igual tráfico; se acepta.

Qué cambiaría la decisión

- Que el costo de Lattice resulte desproporcionado frente al tráfico real.

ADR-018: Ingreso con CloudFront, WAF y ALB sin cambios

Contexto

WAF ya está en uso. El ingreso de clientes es lo más sensible a cambios.

Decisión

Se mantiene CloudFront, WAF y ALB (paso `1`). El uso de ALBs depende de cada servicio; la modernización puede consolidarlos.

Consecuencias

Positivas

- Cero cambio visible para los clientes.

Negativas

- Se heredan las configuraciones actuales hasta que se importen y revisen.

Qué cambiaría la decisión

- Necesidades nuevas de ingreso (por ejemplo, APIs privadas para socios).

ADR-019: AWS Client VPN e IAM Identity Center para el equipo

Contexto

El equipo usa una VPN de proveedor desconocido.

Decisión

AWS Client VPN autenticado con IAM Identity Center para la red (paso `12`), e Identity Center para el acceso a AWS. Se corre en paralelo con la VPN actual hasta el corte.

Consecuencias

Positivas

- Una sola identidad para red y AWS, auditada en CloudTrail.

Negativas

- Costo por conexión y por asociación de subred.

Qué cambiaría la decisión

- Que el acceso a la red deje de ser necesario (todo por SSM Session Manager o Verified Access).

ADR-020: DNS, IPs y direcciones de salida sin cambios

Contexto

Los proveedores de pagos suelen tener en lista blanca las IPs de salida de Kova.

Decisión

No se cambian DNS, IPs públicas ni NAT con Elastic IPs (paso `7`). Al quedarse producción en su cuenta, esto se cumple sin esfuerzo adicional.

Consecuencias

Positivas

- Sin coordinación con proveedores ni riesgo de cortes por listas blancas.

Negativas

- Ninguna relevante.

Qué cambiaría la decisión

- Un cambio de región o de cuenta de producción, que obligaría a coordinar nuevas IPs con cada proveedor.

ADR-021: Llaves de firma de wallets con Nitro Enclaves, KMS y DynamoDB

Contexto

Kova firma transacciones en Stellar (curva Ed25519) para un número alto de wallets. Las opciones de AWS son:

- KMS con una llave asimétrica por wallet: soporta Ed25519 y nunca expone la llave, pero cuesta alrededor de 1 USD por llave al mes, más cada solicitud de firma.
- CloudHSM: soporta Ed25519 (solo en clusters no FIPS), pero un hsm1.medium cuesta 1.60 USD por hora, unos 1,168 USD al mes en us-east-1, y se necesitan al menos dos para alta disponibilidad, 24/7.
- Secrets Manager por llave: costo por secreto al mes, sin aislamiento al usarla.
- Cifrado de sobre: llaves cifradas con una llave de KMS y guardadas en DynamoDB, descifradas solo dentro de un Nitro Enclave.

Decisión

Cifrado de sobre con Nitro Enclaves (pasos `5` y `6`):

1. Cada llave de wallet se guarda cifrada en DynamoDB (o en la base RDS actual, si es donde viven hoy).
2. La llave de KMS que las cifra solo permite `Decrypt` cuando la solicitud trae un documento de atestación del enclave con la imagen esperada (`kms:RecipientAttestation:ImageSha384`).
3. El servicio de firma corre en un grupo de autoescalado de EC2 con Nitro Enclaves. ECS no documenta soporte para enclaves, así que va fuera de ECS.
4. El enclave descifra, firma y descarta la llave; la llave en claro nunca sale del enclave.
5. La llave de KMS es multi-Región y la tabla es global, para el DR (ADR-031).

Es el patrón documentado por AWS para custodia de llaves de blockchain (blog de Web3 sobre Nitro Enclaves) y el recomendado por expertos de AWS para miles de wallets.

Consecuencias

Positivas

- Costo de KMS independiente del número de wallets: una llave y solicitudes de descifrado.
- La llave en claro solo existe dentro del enclave, fuera del alcance del sistema operativo y de los administradores.
- DynamoDB escala sin límites prácticos de llaves.

Negativas

- Agrega EC2 al stack de cómputo (no es Fargate) y un ciclo de despliegue de imágenes de enclave.
- Cambiar la imagen del enclave exige actualizar la política de la llave.

Qué cambiaría la decisión

- Pocas wallets (decenas): KMS con una llave asimétrica por wallet sería más simple.

- Volumen de firmas extremo o un requisito FIPS 140 nivel 3 dedicado: CloudHSM.

ADR-022: Las llaves permanecen en la cuenta de producción

Contexto

Lo ideal sería una cuenta de custodia separada. Pero las llaves de KMS no se pueden mover entre cuentas, y moverlas implicaría reprovisionar producción.

Decisión

Las llaves y el servicio de firma se quedan en la cuenta de producción, aislados por red (Lattice), IAM y la política de la llave.

Consecuencias

Positivas

- Sin riesgo de interrupción por mover llaves.

Negativas

- Menor aislamiento que una cuenta dedicada.

Qué cambiaría la decisión

- Un requisito regulatorio explícito de separar la custodia en otra cuenta.

ADR-023: Origen de las llaves: conservar, importar o generar fuera e importar

Contexto

No se sabe dónde están hoy las llaves de las wallets.

Decisión

- Si ya están en KMS: se conservan.
- Si están fuera de AWS: se importan (KMS admite importar material Ed25519).
- Si no existen: se generan fuera y se importan, para no quedar atados a un proveedor.

Consecuencias

Positivas

- Portabilidad y sin cambios innecesarios.

Negativas

- Generar fuera exige una ceremonia de llaves y custodia del material original.

Qué cambiaría la decisión

- Que no se requiera portabilidad; entonces se generan dentro del enclave.

ADR-024: Secretos de configuración en Secrets Manager y SSM

Contexto

Se asume que Kova ya sigue la práctica recomendada.

Decisión

Secretos de configuración en Secrets Manager (con rotación donde aplique) y parámetros en SSM Parameter Store `SecureString` (paso `8`), cifrados con llaves de KMS por cuenta.

Consecuencias

Positivas

- Sin secretos en código ni en variables de entorno planas.

Negativas

- Costo por secreto en Secrets Manager.

Qué cambiaría la decisión

- Que el inventario muestre secretos fuera de estos servicios; se migran.

ADR-025: CodePipeline y CodeBuild en una cuenta de Deployments

Contexto

Se quiere CI/CD dentro de AWS. El código queda en GitHub (o su host actual). Hoy no hay pipelines: los cambios se hacen a mano.

Opciones consideradas

- Pipelines dentro de cada cuenta de destino.
- Pipelines en la cuenta de AFT.
- Pipelines en una cuenta de Deployments dedicada, como recomienda la guía de AWS para entornos multicuenta.

Decisión

CodePipeline y CodeBuild en una cuenta de Deployments (pasos `07` y `11`), conectados a GitHub con CodeConnections. No se usa CodeCommit. Los pipelines despliegan en cada cuenta asumiendo roles de IaC; ECR replica las imágenes a las cuentas y a la región de DR.

Hay cuatro tipos de pipeline:

Pipeline	Uno por	Etapas
Infraestructura (Terraform)	Cuenta y componente (`infra/live/...`)	Fuente → `fmt`, `validate` y revisión de políticas → `plan` guardado como artefacto → aprobación manual para producción → `apply` con un rol distinto al del `plan`
Aplicación	Servicio de ECS	Fuente → build de la imagen arm64 → pruebas → escaneo de la imagen (ECR con Amazon Inspector) → push a ECR con etiquetas inmutables → despliegue en dev → staging → aprobación → producción con blue/green nativo de ECS (ADR-026)
Imagen del enclave	Servicio de firma	Build de la imagen del enclave → medición del PCR0 → Terraform actualiza la política de la llave de KMS con el nuevo PCR0 → renovación de instancias del grupo de autoescalado; aprobación de dos personas (ADR-021)
Cuentas (AFT)	Solicitud de cuenta	Los administra AFT (ADR-004)

Consecuencias

Positivas

- Todo dentro de AWS y auditado en CloudTrail.
- Los pipelines no tienen credenciales permanentes en las cuentas destino.
- El `plan` revisado es exactamente lo que se aplica.

Negativas

- CodePipeline es menos flexible que otras herramientas de CI.
- Una cuenta más que operar.

Riesgos de seguridad y mitigaciones

La cuenta de Deployments puede asumir los roles que cambian producción, así que pasa a ser el objetivo más valioso de la organización. Quien la controle, o controle un repositorio, puede desplegar en producción.

Riesgo	Mitigación
Compromiso de la cuenta de Deployments	SCPs restrictivas; ninguna persona con escritura; alertas de CloudTrail sobre cambios en pipelines, roles y conexiones
Abuso de los roles de IaC en las cuentas destino	Política de confianza que solo admite el rol de cada pipeline (con condición `aws:SourceAccount`); roles separados de `plan` (solo lectura) y `apply`; permisos mínimos por componente

Riesgo	Mitigación
Cambio malicioso o erróneo desde el repositorio (cadena de suministro)	CodeConnections limitado a repositorios específicos; ramas protegidas y revisión obligatoria en GitHub; aprobación manual antes de producción
Imagen o dependencia comprometida	Escaneo de imágenes, etiquetas inmutables en ECR y, opcionalmente, firma de imágenes
Filtración de artefactos (planes de Terraform con datos sensibles)	Buckets de artefactos cifrados con KMS, acceso solo para los roles del pipeline, ciclo de vida corto
Cambio de la imagen del enclave para robar llaves	El PCR0 solo cambia con aprobación de dos personas y queda registrado en CloudTrail

Qué cambiaría la decisión

- Que el equipo dependa mucho de GitHub Actions; se usaría OIDC hacia los mismos roles de IaC, con las mismas mitigaciones.
- Preferir menos cuentas: los pipelines podrían vivir en la cuenta de AFT, aceptando concentrar más privilegios en ella.

ADR-026: Despliegues blue/green nativos de ECS

Contexto

ECS ahora soporta blue/green de forma nativa, sin CodeDeploy.

Decisión

Despliegues blue/green nativos de ECS (paso `11`), con *circuit breaker* y *rollback* automático. Sin CodeDeploy.

Consecuencias

Positivas

- Menos piezas y configuración en Terraform sin archivos AppSpec.

Negativas

- Según la documentación consultada, el blue/green nativo cambia todo el tráfico de una vez (sin *canary* ni lineal); se revisa si eso cambió.

Qué cambiaría la decisión

- Servicios que requieran *canary*; se usaría CodeDeploy solo para ellos.

ADR-027: OpenTelemetry como capa de desacople de observabilidad

Contexto

Hoy se usa New Relic y se evalúa cambiar de herramienta. Cambiar de herramienta suele implicar re-instrumentar todo.

Decisión

Todo servicio emite logs, métricas y trazas con OpenTelemetry hacia un colector ADOT (paso `9`). New Relic se mantiene durante la migración. Luego se evalúan, con los mismos datos, CloudWatch y CloudWatch Omni, Amazon Managed Grafana con Managed Prometheus, y Datadog como un candidato más.

Consecuencias

Positivas

- Cambiar de herramienta pasa a ser cambiar el destino del colector.
- Permite enviar a dos herramientas en paralelo durante la evaluación.

Negativas

- Instrumentar con OpenTelemetry es un esfuerzo inicial en cada servicio.

Qué cambiaría la decisión

- Nada; la elección de herramienta final queda fuera de esta decisión.

ADR-028: Retención de logs y archivo en S3 con Athena

Contexto

Los logs sin retención crecen sin control y son una de las fuentes de costo que más se escapa.

Decisión

Todo grupo de logs y almacén de trazas tiene retención definida (paso `10`). Los logs que hay que conservar más tiempo se exportan a S3 con ciclo de vida y se consultan con Athena.

Consecuencias

Positivas

- Costo de logs predecible.

Negativas

- Consultar logs viejos es más lento (Athena en lugar de la herramienta de observabilidad).

Qué cambiaría la decisión

- Requisitos regulatorios de retención; ajustan los plazos, no el mecanismo.

ADR-029: FinOps: etiquetado obligatorio, presupuestos y anomalías

Contexto

Sin etiquetas no se sabe qué cuesta cada servicio ni qué es producción.

Decisión

- Etiquetas obligatorias en todo recurso (políticas de etiquetas de Organizations y validación en el pipeline).
- AWS Budgets por cuenta, alertas de facturación y Cost Anomaly Detection.
- Topes de costo en sandboxes y dev.

Consecuencias

Positivas

- Costo por servicio y por entorno visible; detecta gastos inesperados pronto.

Negativas

- Etiquetar lo existente es trabajo manual en la fase de inventario.

Qué cambiaría la decisión

- Nada; es la base para estimar y recortar costos.

ADR-030: CloudTrail organizacional, AWS Config y Security Hub

Contexto

Sin auditoría centralizada no hay forma de saber quién cambió qué ni de detectar desviaciones.

Decisión

CloudTrail organizacional hacia Log Archive, AWS Config para detectar configuraciones incorrectas y drift (con el paquete de conformidad de PCI DSS) y Security Hub en la cuenta Audit (paso `13`). El detalle se deja abierto a propósito.

Consecuencias

Positivas

- Trazabilidad completa y hallazgos centralizados.

Negativas

- Costo de Config por elemento de configuración registrado.

Qué cambiaría la decisión

- Nada; el alcance de reglas se ajusta con el uso.

ADR-031: DR pilot light en una segunda región, sin DRS

Contexto

Se asume RTO de 1 hora (por las dependencias entre sistemas) y RPO de 5 minutos. AWS Elastic Disaster Recovery (DRS) replica servidores con agente; no cubre ECS, Fargate, RDS ni OpenSearch, y AWS recomienda la copia entre regiones de RDS en lugar de DRS para RDS.

Opciones consideradas

- Pilot light: datos replicados siempre, cómputo creado al fallar.
- Warm standby: cómputo mínimo encendido en la segunda región.
- Activo-activo multirregión.

Decisión

Pilot light por costo (paso `14`):

- Réplicas de RDS entre regiones (Aurora Global Database tras la modernización).
- Tabla global de DynamoDB y llave réplica de KMS para las wallets.
- Replicación de ECR.
- Los servicios de ECS, incluido Temporal, se crean con Terraform solo durante una conmutación.

Consecuencias

Positivas

- Cumple RPO de 5 minutos y RTO de 1 hora con el menor costo fijo.

Negativas

- El RTO depende de que el `terraform apply` de la región de DR esté probado; hay que ensayarlo.

Qué cambiaría la decisión

- Un RTO cercano a cero: warm standby o activo-activo, con su costo.

ADR-032: No tocar la infraestructura blockchain

Contexto

Se asume Stellar. No hay información de nodos propios ni de Horizon.

Decisión

No se toca la integración con la red blockchain. Lo único en alcance son las llaves de firma (ADR-021).

Consecuencias

Positivas

- Sin riesgo sobre la pieza más crítica y menos conocida.

Negativas

- Posibles mejoras (nodos propios, redundancia de proveedores RPC) quedan fuera.

Qué cambiaría la decisión

- Contexto sobre cómo se conecta Kova a Stellar hoy.

ADR-033: Reducción del alcance PCI con issuer-processor y segmentación

Contexto

Kova emite tarjetas, así que aplica PCI DSS. El alcance es todo sistema que almacena, procesa o transmite datos de tarjeta, más lo conectado a ellos.

Opciones consideradas

- Kova maneja los datos de tarjeta en su propia infraestructura.
- Un issuer-processor (tipo Pomelo, Marqeta o Galileo) guarda los datos de tarjeta y Kova solo tokens.
- Una cuenta dedicada para el entorno de datos de tarjeta.

Decisión

Issuer-processor más segmentación dentro de la cuenta de producción (paso `4`): los pocos servicios que hablan con el procesador quedan en su propia service network de Lattice, con políticas de IAM, security groups y etiquetas. Paquete de conformidad PCI DSS de AWS Config y el reporte de cumplimiento PCI de AWS (AWS Artifact) para la parte de responsabilidad compartida.

Consecuencias

Positivas

- Alcance PCI mínimo y nada se mueve.

Negativas

- Dependencia del procesador.

Qué cambiaría la decisión

- Que Kova necesite manejar datos de tarjeta directamente; se evaluaría una cuenta dedicada como paso de modernización.